Exercises: Introductory Course on Implicitly Switched ODEs and Filippov Sliding

Andreas Sommer, Lev Gromov, Marvin Hertweck, Pilar Keller

July 09, 2026

Task 4: Solution

a) The maximum size of the population approaches $C = 1$ asymptotically. The population grows fastest when $P(t) = \frac{C}{2}$.

File: `rhsTask4.m`

```
function dy = rhsTask4(~, y, p)
r = 2;
C = 1;

P = y(1);

dP = r.*P .* (1 - P./C);
dy = dP;
end
```

File: `runTask4.m`

```
%% Subtask a) and b)
%% Setup problem
tspan = [0, 5];
y0 = 0.1;

T = 0.5;
alpha = 0.5;
p = [T, alpha];

rhs = @rhsTask4;
integrator = @ode45;
optsIntegrator = odeset('AbsTol', 1e-10, 'RelTol', 1e-8);
datahandle = prepareDatahandleForIntegration(rhs, 'integrator', integrator, '
    options', optsIntegrator);

%% Solve
solution = solveODE(datahandle, tspan, y0, p);

%% Plot solution
plotPopulation(solution);
```

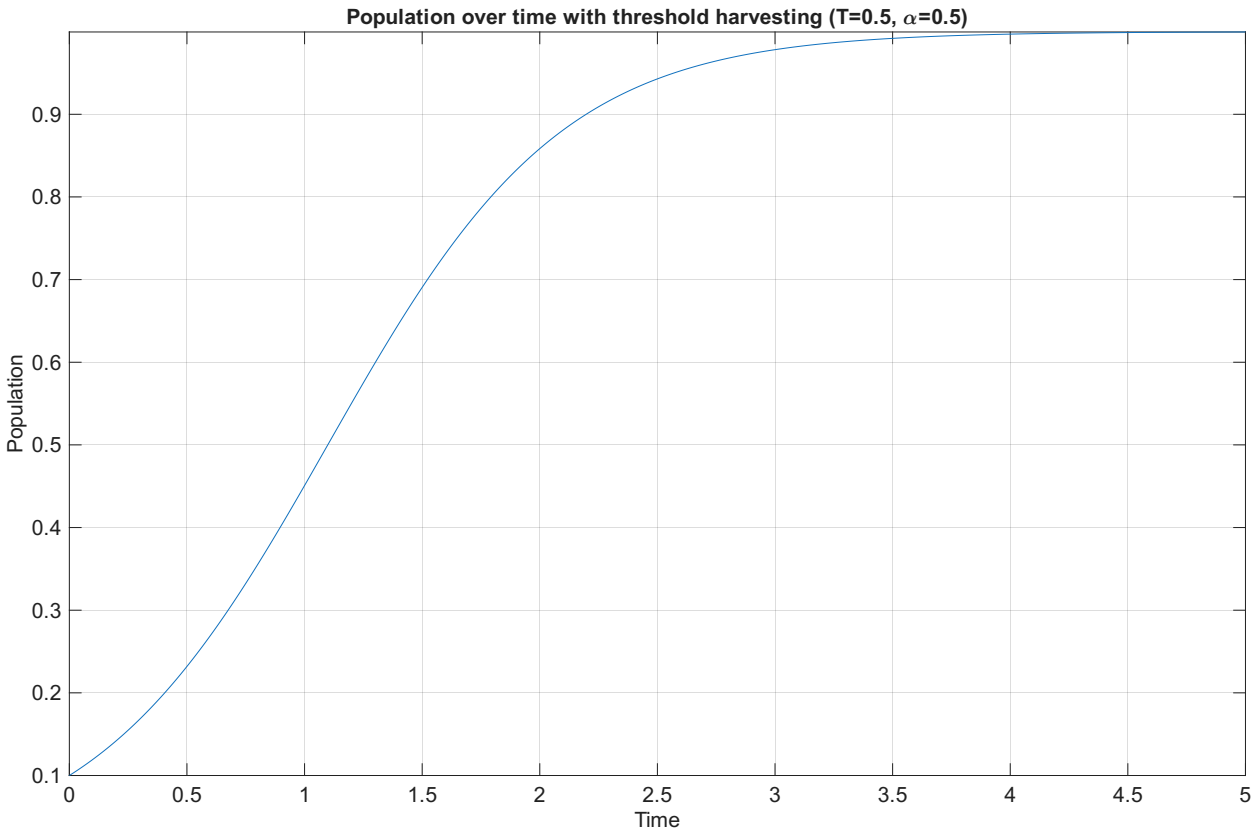

Figure 1: Population plot for subtask a)

b) If the parameters are set such that an actual harvest occurs, then the maximum population is equal to the harvest threshold T. Otherwise, the maximum size of the population approaches $C = 1$ asymptotically as before.

File: `rhsTask4.m`

```
function dy = rhsTask4(~, y, p)
r = 2;
C = 1;

T     = p(1);
alpha = p(2);
P = y(1);

dP = r.*P .* (1 - P./C);
dH = 0;
dy = [dP; dH];

harvestCondition = P - T;
if ifdiff_jumpif(harvestCondition, 1)
    harvest = alpha .* P;
    jump = [-harvest; harvest];
    ifdiff_update(jump);
end
end
```

File: `runTask4.m`

```
%% Subtask a) and b)
%% Setup problem
tspan = [0, 5];
y0 = [0.1; 0];

T = 0.5;
alpha = 0.5;
p = [T, alpha];

rhs = @rhsTask4;
integrator = @ode45;
optsIntegrator = odeset('AbsTol', 1e-10, 'RelTol', 1e-8);
datahandle = prepareDatahandleForIntegration(rhs, 'integrator', integrator, '
    options', optsIntegrator);

%% Solve
solution = solveODE(datahandle, tspan, y0, p);

%% Plot solution
plotPopulation(solution);
```

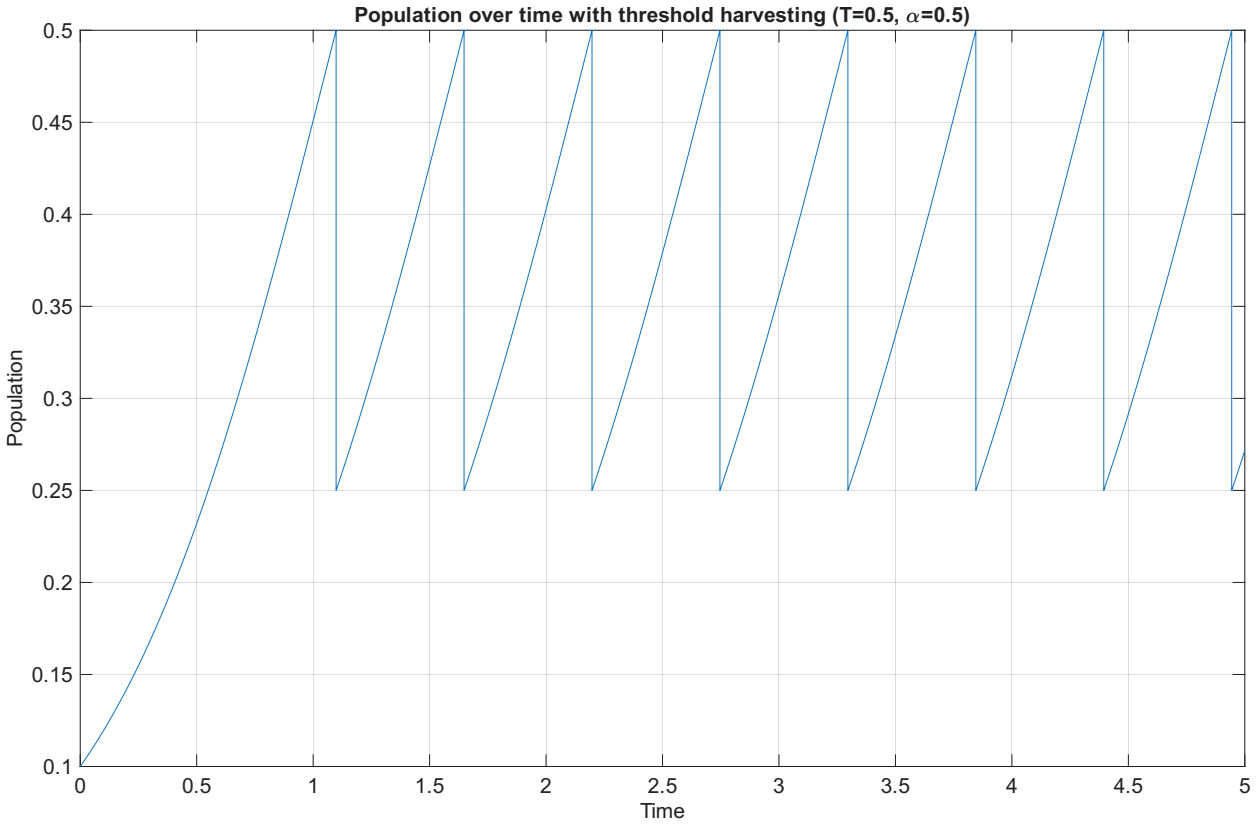

Figure 2: Population plot for subtask b)

c) If $T = 0$, then a harvest will never trigger because the population starts at $P(0) = 0.1 > T = 0$ and thus never crosses the threshold $T = 0$. Similarly, if $T = C$, then a harvest also never triggers because the population merely approaches $T = C$ asymptotically, but may never reach and cross it.

If $\alpha = 0$, then harvests may or may not trigger depending on the value of T, but will never have any

effect on the solution because the proportion of the population extracted from the pond is always zero.

If $\alpha = 1$ and a harvest occurs (i.e. if $P(0) < T < C$), then the entire population will be depleted at the first harvest. The population remains at zero from that point onward.

File: `runTask4.m`

```
%% Subtask c)
%% Solve for multiple parameters
solutionFunc = @(p) solveODE(datahandle, tspan, y0, p);

solutionGrid = solveOnGrid(solutionFunc);

%% Plot solution for multiple parameters
plotPopulationGrid(solutionGrid);
```

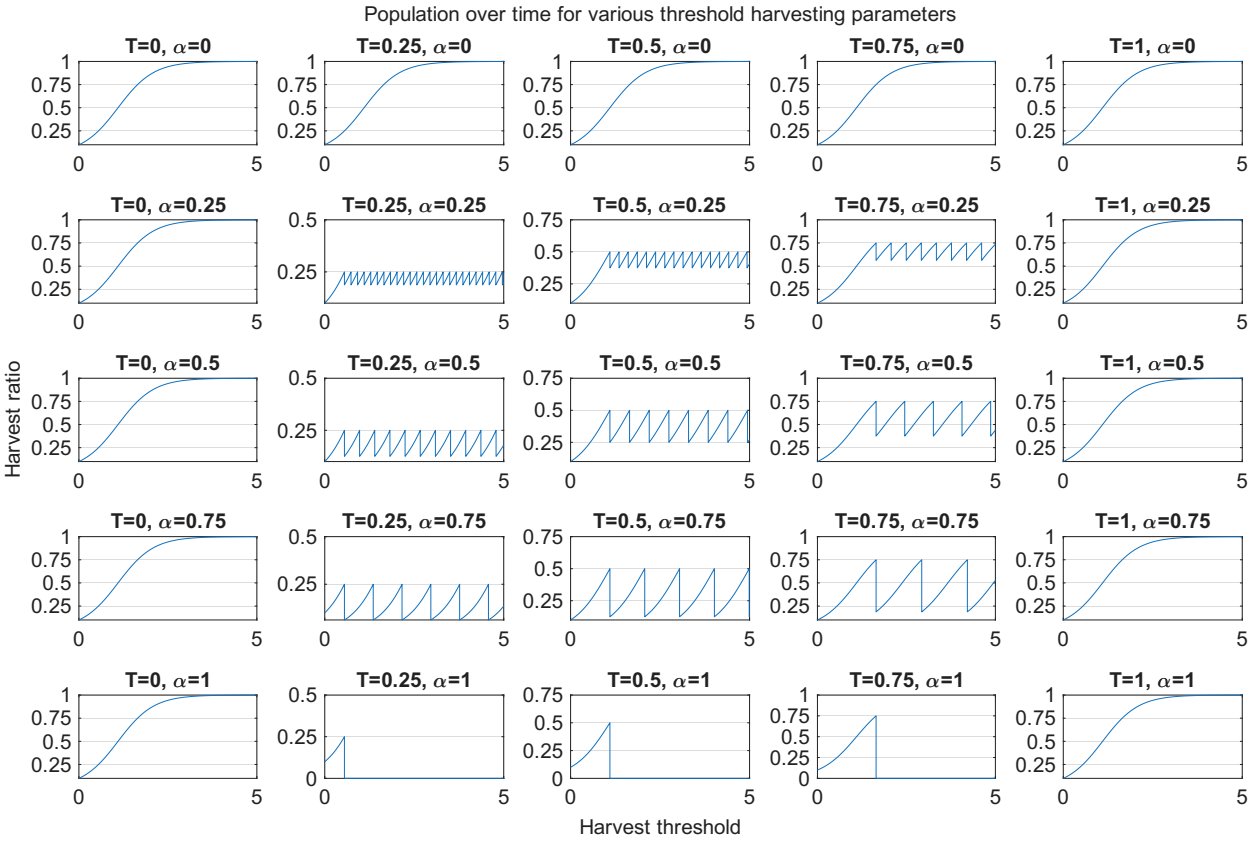

Figure 3: Population grid plot for subtask c)

d) If $T = 0$, $T = C$ or $\alpha = 0$, then no actual harvests occur, so the objective function is determined only by the population at the end of the time interval, which will be approximately equal to $C = 1$.

If $\alpha = 1$ and a harvest occurs (i.e. if $P(0) < T < C$), then the entire population will be depleted at the first harvest, so the objective function is determined only by the amount harvested then, which will be equal to T, since $P(t) = T$ at the time of the harvest.

The parameters which maximimze the objective function on the grid are $T = 0.5$ and $\alpha = 0.25$.

File: `runTask4.m`

```
%% Subtask d)
%% Compute objective
objectiveFunc = @(solution) sum(solution.y(:, end));

objectiveGrid = arrayfun(objectiveFunc, solutionGrid);

%% Plot objective
plotObjective(solutionGrid, objectiveGrid);
```

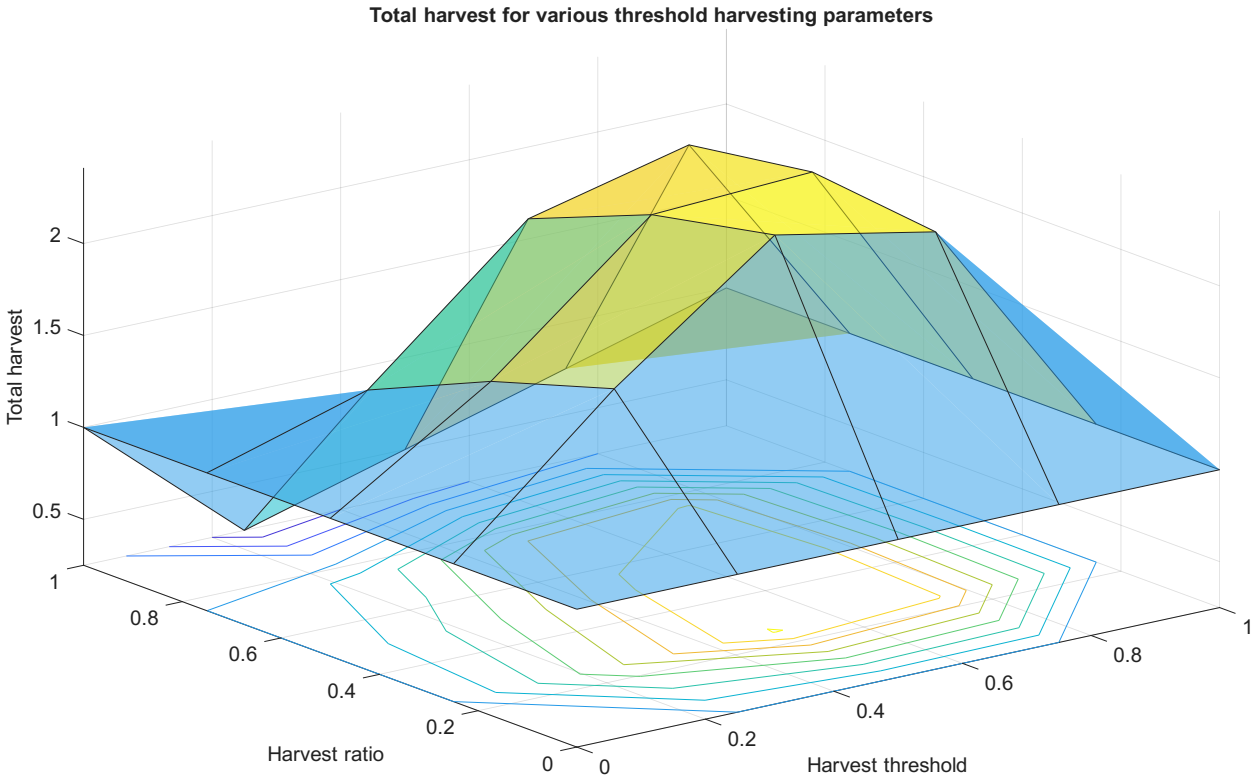

Figure 4: Objective function plot for subtask d)

e) At the parameters which maximize the objective function on the grid, namely $T = 0.5$ and $\alpha = 0.25$, the gradient approximately points in the direction $d = (0.07, -0.02)$, which implies that we may achieve an even bigger value for the objective function by increasing the harvest threshold from $T = 0.5$ and reducing the harvest ratio from $\alpha = 0.25$.

File: `runTask4.m`

```
%% Subtask e)
%% Compute gradient
function g = computeGradient(datahandle, solution)
sensitivity = generateSensitivityFunction(datahandle, solution, 'calcGy', false);
Gp = sensitivity(solution.x(end)).Gp;
dP = Gp(1, :);
dH = Gp(2, :);
g = dP + dH;
end
gradientFunc = @(solution) computeGradient(datahandle, solution);
```

```
gradientGrid = arrayfun(gradientFunc, solutionGrid, 'UniformOutput', false);

%% Plot gradient
plotGradient(solutionGrid, objectiveGrid, gradientGrid)
```

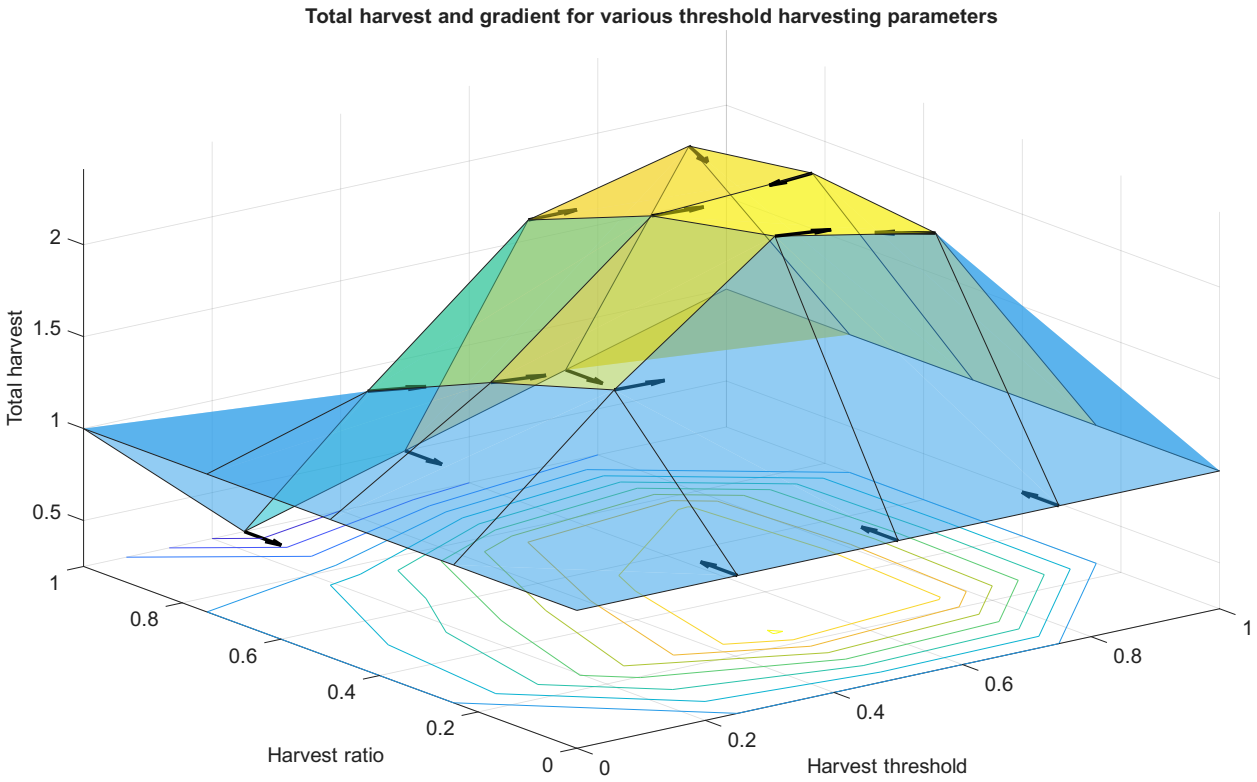

Figure 5: Gradient plot for subtask e)

f) After the first harvest, the population is strictly bound to the interval $J = [T, T - \alpha T]$. The midpoint of this interval is $m = T - \frac{(T-(T-\alpha T))}{2} = T(1 - \frac{\alpha}{2})$. For the optimal parameters $T^* \approx 0.56$ and $\alpha^* \approx 0.2$ the midpoint is located at approximately $m^* \approx 0.5 = \frac{C}{2}$.

This behavior is not coincidental: Each fish produced over the time interval will be added to the value of the objective function, either as part of a harvest, or at the end of the time interval when the pond is emptied. In other words, fish that have been produced can never disappear. Therefore, to maximize the total amount of fish produced, it is sufficient to maximize the population growth over the entire time interval. The population growth is maximal when the population is at half of the total capacity of the pond, i.e. $P(t) = \frac{C}{2}$. The population growth decreases symmetrically as the population distances itself from the optimal point. Thus, our goal is to keep the population as close to the optimal point $\frac{C}{2}$ as possible.

We know that our choice of parameters T and α will constrain the population to the interval $J = [T, T - \alpha T]$. Since the population growth decreases quadratically with the distance of the population from the optimal point, we should choose T and α, s.t. the interval J has minimal length and is centered at $\frac{C}{2}$. Thus, choosing $\alpha^* = 0.2$ will minimize the length of the interval (under the constraint $\alpha \geq 0.2$). The optimal harvest threshold T^* can then be obtained by enforcing $m^* = T^*(1 - \frac{\alpha^*}{2}) = \frac{C}{2}$ which yields $T^* = \frac{C}{2-\alpha^*} \approx 0.56$. This matches the numerical results obtained by the optimizer.

File: `runTask4.m`

```
%% Subtask f)
%% Optimize
function [f, g] = objWithGrad(p, solutionFunc, objectiveFunc, gradientFunc)
solution = solutionFunc(p);
f = -objectiveFunc(solution);
if nargout > 1
    g = -gradientFunc(solution);
end
end
optimizationFunc = @(p) objWithGrad(p, solutionFunc, objectiveFunc, gradientFunc);

[pOpt, objOpt] = optimizeParameters(optimizationFunc);
solutionOpt = solutionFunc(pOpt);

%% Plot optimal solution
fprintf('Optimal Parameters: T=%g, alpha=%g\n', pOpt);
fprintf('Optimal Total Harvest: %g\n', objOpt);

plotPopulation(solutionOpt);
```

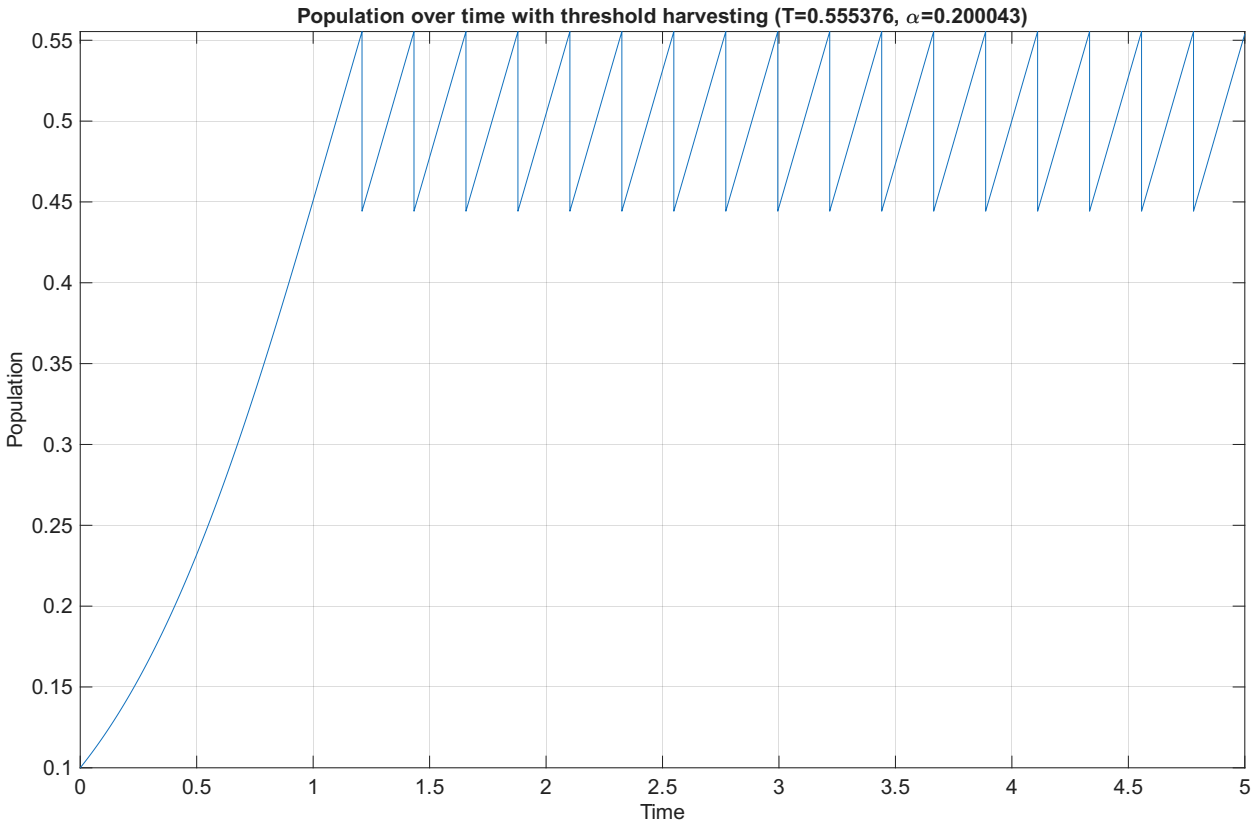

Figure 6: Gradient plot for subtask f)